



SMART FLOOD LEVEL MONITORING SYSTEM USING ULTRASONIC SENSOR

EXERCISE NO – 01

HARRISH T M	– 24MER016
KAPIL P	– 24MER022
RAMPRASATH D	– 24MER047
SARAVANAKARTHIK S R	– 24MER055
SETHUPATHI M	– 24MER056

1. SMART FLOOD LEVEL MONITORING SYSTEM USING ULTRASONIC SENSOR

1. Project overview:

Flooding is one of the major natural disasters that can cause damage to buildings, roads, agricultural lands and human life. Early detection of increasing water levels can help people take necessary safety measures.

The proposed project is a Smart Flood Level Monitoring System using an Ultrasonic Sensor. The system uses an HC-SR04 ultrasonic sensor to continuously measure the distance between the sensor and the water surface. An ESP8266 NodeMCU processes the sensor data and calculates the water level.

The system provides local alerts using a green LED, red LED and buzzer. The ESP8266 can also connect to Wi-Fi and send the water-level data to Adafruit IO, where the level and flood status can be monitored through a dashboard.

2. Problem Statement:

Traditional flood monitoring systems may require manual observation or expensive monitoring equipment. Manual monitoring can be slow and may not provide sufficient warning when the water level rises rapidly.

Therefore, there is a need for a simple and low-cost system that can:

1. Continuously monitor water level.
2. Detect increasing water levels automatically.
3. Provide an immediate local warning.
4. Send the measured data to a remote dashboard.
5. Help in early flood-condition identification.

The proposed system addresses these requirements using an HC-SR04 ultrasonic sensor, ESP8266, LEDs, buzzer and Adafruit IO.

3.Proposed Solution:

The proposed system uses an HC-SR04 ultrasonic sensor mounted above the water surface. The ultrasonic sensor sends an ultrasonic pulse toward the water surface. The pulse reflects from the water surface and returns to the sensor. The ESP8266 measures the echo time and calculates the distance.

The water level is calculated using:

Water Level = Tank Height – Measured Distance

For a 30 cm prototype tank:

Water Level Percentage = (Water Level / 30) × 100

The system uses the following conditions:

Water level	Condition	Green LED	Red LED	Buzzer
Below 60%	Normal	ON	OFF	OFF
60% to below 75%	Warning	OFF	ON	OFF
75% and above	Critical Flood	OFF	ON	ON

4.System Architecture:

The proposed flood monitoring system consists of four main sections: sensing, processing, alerting, and remote monitoring.

The HC-SR04 ultrasonic sensor acts as the sensing unit and measures the distance between the sensor and the water surface. This information is given to the ESP8266 NodeMCU, which acts as the main processing unit. The ESP8266 calculates the water level and compares it with predefined safety limits.

Based on the detected water level, the system provides local alerts using a green LED, red LED, and buzzer. The green LED indicates a normal water level, while the red LED indicates a high or critical water level. The buzzer is activated when the water level reaches the critical condition.

For remote monitoring, the ESP8266 connects to the Internet through **Wi-Fi** and sends the measured data to **Adafruit IO**. The dashboard displays the water level, sensor distance, and flood status.

5.Circuit Table:

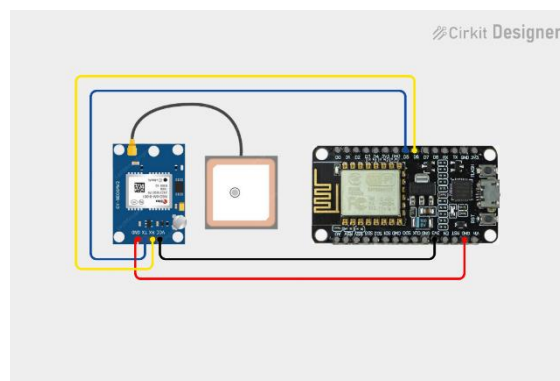
S. No.	Component	ESP8266 Pin / Connection
1	HC-SR04 TRIG	GPIO14
2	HC-SR04 ECHO	GPIO12 (through voltage divider)
3	Green LED	GPIO5
4	Red LED	GPIO4
5	Buzzer	GPIO0 (LOW level trigger)
6	HC-SR04 VCC	VIN / 5V
7	HC-SR04 GND	GND
8	LED Negative Terminals	GND
9	Buzzer Negative Terminal	GND

6. Circuit Design:

The For your report, you can use this alternative circuit design with the NodeMCU pin names D1, D2, D3, D5 and D6, along with the corresponding GPIO numbers.

Connections:

- HC-SR04 TRIG → **D5 (GPIO14)**
- HC-SR04 ECHO → **D6 (GPIO12)** through voltage divider
- Green LED → **D1 (GPIO5)** + 220 Ω resistor
- Red LED → **D2 (GPIO4)** + 220 Ω resistor
- Low-level-trigger buzzer → **D3 (GPIO0)**
- HC-SR04 VCC → **VIN/5V**
- All GND terminals → **GND**



7. Algorithm:

Step 1: Start

Initialize the ESP8266, ultrasonic sensor, LEDs and buzzer.

Step 2: Initialize Parameters

Set the tank height to 30 cm and define the flood-level limits:

- Normal: $< 60\%$
- Warning: $60\% - 74\%$
- Critical: $\geq 75\%$

Step 3: Connect to Network

Connect the ESP8266 to the configured Wi-Fi network and establish a connection with Adafruit IO.

Step 4: Measure Water Distance

Send a $10\ \mu\text{s}$ trigger pulse from the ESP8266 to the HC-SR04 sensor.

Step 5: Receive Echo

Measure the time taken for the ultrasonic wave to return from the water surface.

Step 6: Calculate Distance

Calculate the distance between the sensor and water surface using:

$$\text{Distance} = (\text{Echo Time} \times 0.0343) / 2$$

Step 7: Calculate Water Level

Calculate the actual water height:

$$\text{Water Level} = \text{Tank Height} - \text{Distance}$$

Step 8: Calculate Percentage

Convert the water level into percentage:

$$\text{Level \%} = (\text{Water Level} / \text{Tank Height}) \times 100$$

Limit the result between 0% and 100%.

Step 9: Determine Flood Condition

Compare the calculated percentage with the predefined limits.

IF Level < 60%

Normal condition

Green LED ON

Red LED OFF

Buzzer OFF

ELSE IF Level < 75%

Warning condition

Green LED OFF

Red LED ON

Buzzer OFF

ELSE

Critical flood condition

Green LED OFF

Red LED ON

Buzzer ON

Step 10: Update IoT Dashboard

Send the following data to Adafruit IO:

- Water level percentage
- Sensor distance
- Flood status

Step 11: Display Information

Display the sensor reading, water level, percentage and status on the Serial Monitor.

Step 12: Repeat

Wait for a short interval and repeat the measurement continuously.

8. Coding:

```
#include "AdafruitIO_WiFi.h"
#define WIFI_SSID    "Harrish"
#define WIFI_PASSWORD "Password"
#define IO_USERNAME  "saravanakarthiks"
#define IO_KEY       "aio_idPg82y91fmNVXL0iJA71zKwH97q"
AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASSWORD);
AdafruitIO_Feed *waterLevelFeed = io.feed("water-level");
AdafruitIO_Feed *distanceFeed  = io.feed("distance");
AdafruitIO_Feed *statusFeed    = io.feed("flood-status");
#define TRIG_PIN  14
#define ECHO_PIN  12
#define GREEN_LED  5
#define RED_LED   4
#define BUZZER    0
const float TANK_HEIGHT = 30.0; // Tank height in cm
#define BUZZER_ON  LOW
#define BUZZER_OFF HIGH
unsigned long lastSendTime = 0;
const unsigned long SEND_INTERVAL = 5000; // 5 seconds
void setup() {
  Serial.begin(115200);
  delay(1000);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(BUZZER, OUTPUT);
  digitalWrite(TRIG_PIN, LOW);
  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, LOW);
  digitalWrite(BUZZER, BUZZER_OFF);
  Serial.println();
  Serial.println()
  Serial.println(" SMART FLOOD LEVEL MONITORING");
  Serial.println(" ESP8266 + HC-SR04");
  Serial.println()
  Serial.println("Connecting to Adafruit IO...");
  io.connect();
  while (io.status() < AIO_CONNECTED) {
    Serial.print(".");
    delay(500);
```

```

}
Serial.println();
Serial.println("Adafruit IO Connected!");
Serial.println();
}
void loop() {
  io.run();
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  long duration = pulseIn(ECHO_PIN, HIGH, 30000);
  if (duration == 0) {
    Serial.println("ERROR: No ultrasonic signal!");
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, LOW);
    digitalWrite(BUZZER, BUZZER_OFF);
    delay(1000);
    return;
  }
  float distance = duration * 0.0343 / 2.0;
  float waterLevel = TANK_HEIGHT - distance;
  float levelPercentage =
    (waterLevel / TANK_HEIGHT) * 100.0;
  if (levelPercentage < 0) {
    levelPercentage = 0;
  }
  if (levelPercentage > 100) {
    levelPercentage = 100;
  }

  String floodStatus;
  if (levelPercentage < 60) {
    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(RED_LED, LOW);
    digitalWrite(BUZZER, BUZZER_OFF);
    floodStatus = "NORMAL";
    Serial.println("STATUS: NORMAL");
  }
  else if (levelPercentage < 75) {
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, HIGH);
    digitalWrite(BUZZER, BUZZER_OFF);

```

```

    floodStatus = "WARNING";
    Serial.println("STATUS: WARNING - HIGH WATER LEVEL");
}
else {
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(REDA_LED, HIGH);
    digitalWrite(BUZZER, BUZZER_ON);
    floodStatus = "CRITICAL FLOOD";
    Serial.println("STATUS: CRITICAL FLOOD!");
    delay(300);
    digitalWrite(BUZZER, BUZZER_OFF);
}
Serial.print("Distance    : ");
Serial.print(distance);
Serial.println(" cm");
Serial.print("Water Level  : ");
Serial.print(waterLevel);
Serial.println(" cm");
Serial.print("Level        : ");
Serial.print(levelPercentage);
Serial.println(" %");
Serial.print("Flood Status : ");
Serial.println(floodStatus);
Serial.println();
if (millis() - lastSendTime >= SEND_INTERVAL) {
    lastSendTime = millis();
    Serial.println("Sending data to Adafruit IO...");
    waterLevelFeed->save(levelPercentage);
    distanceFeed->save(distance);
    statusFeed->save(floodStatus);
    Serial.println("Data sent successfully!");
    Serial.println();
}
delay(1000);
}

```

9. System Working:

When the system is switched ON, the ESP8266 initializes the ultrasonic sensor, LEDs and buzzer. If IoT monitoring is enabled, the ESP8266 also connects to the configured Wi-Fi network and Adafruit IO.

The HC-SR04 sensor is positioned above the water surface. It continuously measures the distance to the water.

For example, if the tank height is 30 cm and the measured distance is 15 cm:

$$\text{Water Level} = 30 - 15$$

$$\text{Water Level} = 15 \text{ cm}$$

Percentage:

$$\text{Water Level (\%)} = (15 / 30) \times 100$$

$$\text{Water Level} = 50\%$$

Since 50% is below 60%, the system considers the condition normal.

Normal condition

$$\text{Water Level} < 60\%$$

Green LED → ON

Red LED → OFF

Buzzer → OFF

Status → NORMAL

Warning condition

$$60\% \leq \text{Water Level} < 75\%$$

Green LED → OFF

Red LED → ON

Buzzer → OFF

Status → WARNING

Critical condition

$$\text{Water Level} \geq 75\%$$

Green LED → OFF

Red LED → ON

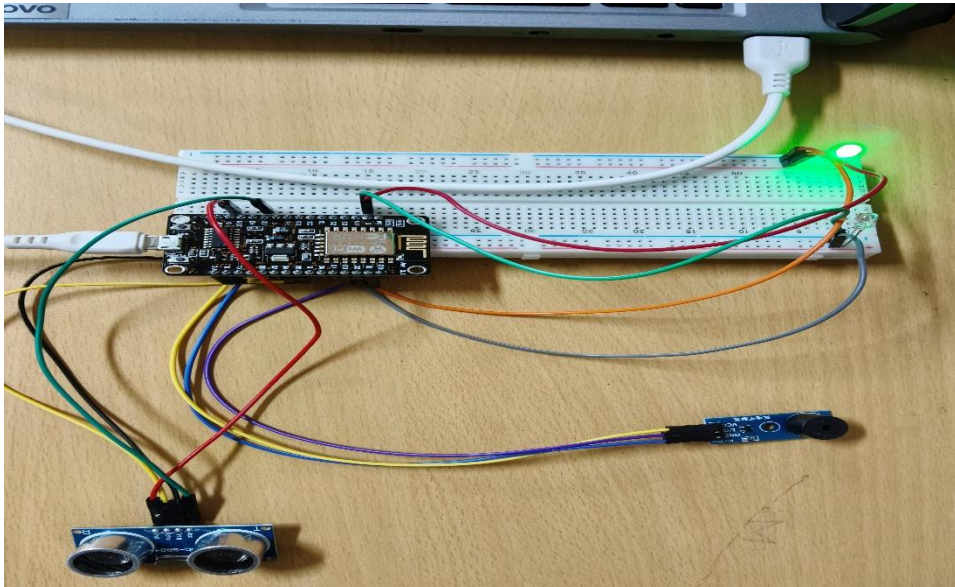
Buzzer → ON

Status → CRITICAL FLOOD The water-level information is also displayed on the Adafruit IO dashboard when Wi-Fi and Internet connectivity are available.

10. Bill of Materials:

COMPONENT	QUANTITY
ESP8266	1
HC-SR04 Ultrasonic Sensor	1
Buzzer	1
LED Indicators	2
LCD Display	1
Breadboard	1
Jumper Wires	Required

11. Prototype Implementation:



12.Results & Performance Analysis:

1. System Implementation

- The Smart Flood Level Monitoring System was successfully designed and implemented using ESP8266 and HC-SR04 ultrasonic sensor.
- The system continuously monitors the water level and calculates the corresponding water-level percentage.
- The system provides automatic alerts based on the detected water level.

2. Normal Condition

- When the water level is below 60%, the system indicates a Normal condition.
- Green LED → ON
- Red LED → OFF
- Buzzer → OFF
- The system continues normal water-level monitoring.

3. Warning Condition

- When the water level reaches 60% to below 75%, the system indicates a Warning condition.
- Green LED → OFF
- Red LED → ON
- Buzzer → OFF
- This indicates that the water level is increasing and requires attention.

4. Critical Flood Condition

- When the water level reaches 75% or above, the system detects a Critical Flood condition.
- Green LED → OFF
- Red LED → ON
- Buzzer → ON
- The buzzer provides an immediate local warning.

5. Ultrasonic Measurement

- The HC-SR04 provides non-contact water-level measurement.
- The sensor continuously measures the distance between the sensor and the water surface.
- As the water level increases, the measured distance decreases.

6. IoT Monitoring

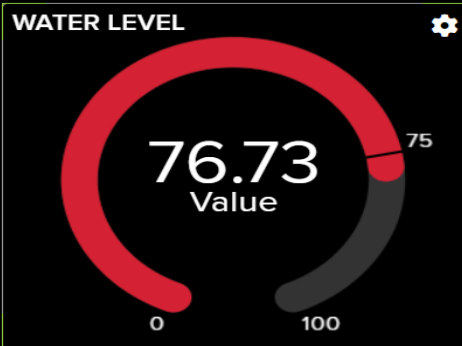
- The ESP8266 connects to the Internet through Wi-Fi.
- The measured data is sent to the Adafruit IO dashboard.
- The dashboard displays:
 - Water level
 - Sensor distance
 - Flood status

7. Overall Performance

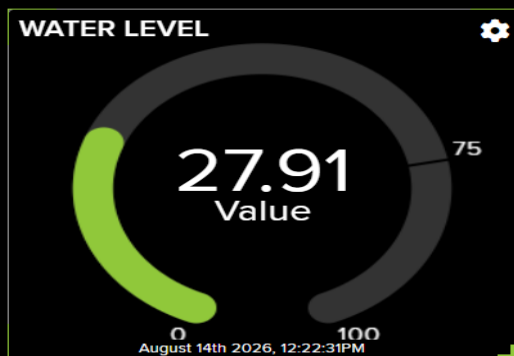
- The system provides real-time local and remote flood monitoring.
- The LED and buzzer alerts operate automatically according to the water level.
- The proposed system is low-cost, simple and suitable for prototype-level flood monitoring applications.
- The combination of HC-SR04, ESP8266, LEDs, buzzer, Wi-Fi and Adafruit IO provides an effective early-warning system for rising water levels.

SERIAL MONITOR OUTPUT:

```
COM5
Sending data to Adafruit IO...
Data sent successfully!
-----
STATUS: NORMAL
Distance      : 20.75 cm
Water Level   : 9.25 cm
Level         : 30.83 %
Flood Status  : NORMAL
-----
STATUS: NORMAL
Distance      : 21.28 cm
Water Level   : 8.72 cm
Level         : 29.06 %
Flood Status  : NORMAL
-----
STATUS: NORMAL
Distance      : 21.97 cm
Water Level   : 8.03 cm
Level         : 26.77 %
Flood Status  : NORMAL
-----
STATUS: NORMAL
Distance      : 20.65 cm
Water Level   : 9.35 cm
Level         : 31.17 %
Flood Status  : NORMAL
-----
STATUS: NORMAL
Distance      : 21.63 cm
Water Level   : 8.37 cm
Level         : 27.91 %
Flood Status  : NORMAL
-----
Sending data to Adafruit IO...
Data sent successfully!
-----
STATUS: NORMAL
Distance      : 20.58 cm
Water Level   : 9.42 cm
Level         : 31.40 %
Flood Status  : NORMAL
-----
STATUS: NORMAL
Distance      : 20.94 cm
Water Level   : 9.06 cm
Level         : 30.20 %
Flood Status  : NORMAL
-----
STATUS: NORMAL
Distance      : 21.81 cm
Water Level   : 8.19 cm
Level         : 27.28 %
Flood Status  : NORMAL
-----
☒ Autoscroll ☐ Show timestamp
```

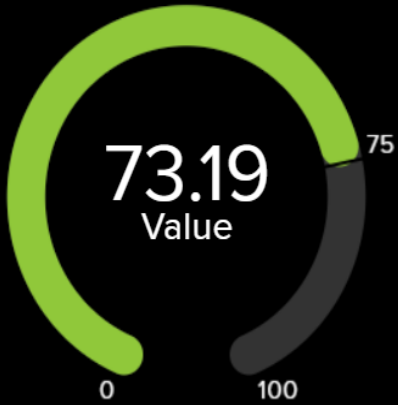


FLOOD STATL ⚙️
CRITICAL FLOOD

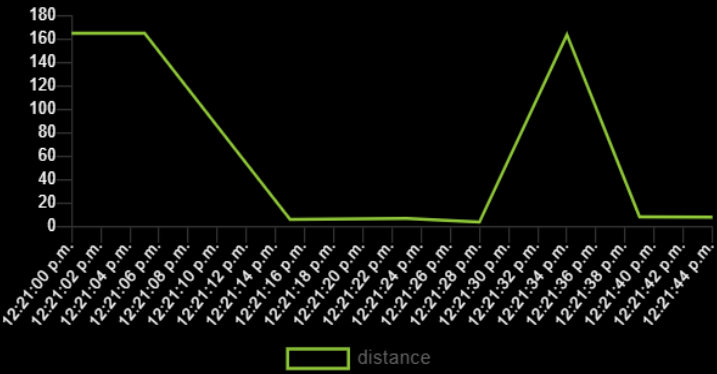


FLOOD STATL ⚙️
NORMAL

WATER LEVEL



DISTANCE



FLOOD STATU



WARNING